

VAJRA

Verified Payments on Monad

Product Requirements Document (PRD)
Technical Requirements Document (TRD)
Security Architecture and Delivery Blueprint

Design system intentionally excluded

The coding agent must stop and request a frontend design discussion before visual implementation.

Document version	1.0 - Build-ready specification
Product	Vajra
Selected domain	vajra.xyz (ownership/availability must be confirmed before launch)
Target network	Monad test environment first; mainnet explicitly out of scope
Primary release	Native MON, recipient-authenticated one-time payments
Status	Specification frozen for functional and security implementation; visual design pending

Core promise: Signed by them. Final on Monad.

1. Document Control and Reading Guide

This document is the authoritative product, technical and security blueprint for Vajra v1. It is written for product owners, coding agents, smart-contract engineers, frontend engineers, QA engineers, reviewers and hackathon judges. It deliberately excludes the application visual design system because that decision has not yet been made.

NON-NEGOTIABLE DESIGN GATE No Design.md, visual design system, brand palette, typography system, animation language, landing-page composition or component styling may be created from this document. The coding agent may build a semantic functional skeleton only, then must stop and ask the product owner for a dedicated frontend design discussion before visual implementation.

Item	Decision
Authoritative product name	Vajra
Working domain	vajra.xyz; confirm control before public references
Initial asset	Native MON only
Primary authentication	Registered device passkey using WebAuthn/P-256
Fallback authentication	Recipient wallet EIP-712 / ERC-1271 signature
Settlement	Exact one-time atomic transfer to recipient
Finality experience	Observe real Monad inclusion/finality; never hardcode elapsed time
Contract governance	Immutable, non-upgradeable, no owner, no fee, no admin withdrawal
Visual design	Pending separate owner-approved design process

How to use this blueprint

1. Freeze product and security scope before coding.
2. Implement and test the contracts before building transaction UI.
3. Build only a minimal semantic functional frontend until all core flows are stable.
4. Stop at the frontend design gate and request owner direction.
5. Resume visual implementation only after written design approval.
6. Do not represent testnet readiness as mainnet readiness.

Normative language

The words MUST, MUST NOT, SHOULD, SHOULD NOT and MAY are used as requirements. MUST-level requirements are release blockers. SHOULD-level requirements may be deferred only with a documented reason and explicit product-owner approval.

2. Executive Summary

Vajra is a recipient-authenticated payment handshake for Monad. A recipient creates exact payment terms, authenticates those terms with a registered device passkey or wallet signature, and shares a link or QR code. A sender opens the request, verifies the signed terms without pasting an address, pays the exact native-MON amount once, and receives a permanent settlement receipt after the transaction is finalized.

ONE SENTENCE Vajra lets the recipient authenticate the exact amount, recipient, expiry and optional payer restriction before the sender pays, preventing silent tampering and duplicate settlement.

Why this product exists

- Wallet-address copy/paste is error-prone and vulnerable to social engineering, address poisoning and complete-link substitution.
- Traditional payment links often treat amount and recipient metadata as editable convenience fields rather than recipient-authenticated terms.
- Users struggle to distinguish wallet signature, transaction broadcast, block inclusion and irreversible finality.
- Passkeys can provide a more human request-creation experience without granting authority to spend the recipient wallet balance.

What makes Vajra technically distinctive

Capability	Product effect
WebAuthn/P-256 request authentication	Recipient can authorize a payment request with Face ID, Touch ID, Windows Hello, device PIN or a security key after one-time registration.
Exact single-use settlement	The first valid payment succeeds; a second attempt fails at contract level.
Domain-separated request digest	The same request cannot be replayed on another chain or another Vajra deployment.
Visible Monad finality	The receipt distinguishes wallet approval, broadcast, inclusion and finalization using observed chain state.
Non-custodial atomic transfer	Vajra does not maintain user balances; successful payment is transferred directly to the signed recipient.
No privileged administrator	No owner can redirect, seize, pause or upgrade payment logic.

Primary judge demo

1. Recipient registers a device passkey with a Monad wallet.
2. Recipient creates a 0.01 MON request and authenticates it with the device passkey.
3. Sender opens the link and verifies the exact terms before connecting a wallet.
4. Sender pays and watches the real transaction move from broadcast to finality.
5. The receipt becomes permanently sealed from real event and state data.
6. A modified amount fails signature verification.
7. A second payment attempt fails onchain because the request is already consumed.

3. Product Definition

Product statement

Vajra is a standalone Monad payment application for direct person-to-person payments. It is not a wallet, escrow, exchange, merchant acquiring network, subscription service, marketplace or social platform.

Core promise

Pay the intended recipient under terms the recipient authenticated before payment, then obtain a verifiable final receipt.

Primary users

User	Need	Vajra outcome
Recipient	Request an exact payment without repeatedly sharing and explaining a raw address.	Creates a signed, expiring, shareable request.
Sender	Know that recipient, amount, network and expiry were not changed in transit.	Verifies the request before paying.
Returning contact	Recognize continuity with a previously used wallet address.	Sees local contact continuity without treating it as legal identity.
Reviewer or judge	Verify the feature is real rather than a local success toast.	Can inspect transaction, event, request state and duplicate rejection.

Product principles

- **One real action over many fake features:** A single secure payment handshake must be complete end to end.
- **No pasted destination:** The sender must not be required to paste a recipient address into the payment flow.
- **No silent mutation:** Changing any authenticated field invalidates the request.
- **No fake success:** Success requires a successful receipt, matching event and paid onchain state.
- **No hidden custody:** The contract must not expose user deposit balances or administrative withdrawal paths.
- **No fake identity claims:** A signature proves wallet or credential control, not the real-world identity of a person.
- **No chain decoration:** Monad-specific capabilities must materially improve the product experience.

Product name and language

The product name is Vajra. Feature labels may use functional terms such as Vajra Request, Vajra Touch, Vajra Code and Vajra Seal. These names are product vocabulary, not a visual-design decision.

4. Problem Definition and Success Metrics

User problem

A sender who receives a wallet address and amount through chat must trust that the destination, amount and network were copied correctly and not substituted. Even after broadcasting, the sender may not know whether the payment merely entered a mempool, was included in a block or became final. The recipient must repeatedly open a wallet and sign messages or transactions to create trustworthy requests.

Target outcome

A recipient should be able to create a request in seconds using a device credential, while a sender should be able to verify every material term before connecting a wallet and complete a one-time payment with a verifiable final receipt.

V1 success metrics

Metric	Target	Reason
Request creation	Under 45 seconds after passkey setup	Fast enough for person-to-person use.
Request verification	All authenticated fields visible before wallet connection	Trust before action.
Payment attempts	Second fulfillment rejected by contract	Proves real state and replay protection.
Tamper test	Any signed-field mutation invalidates request	Core product guarantee.
False success paths	Zero	Required for trust and judging.
Contract privileges	Zero privileged payment controls	Reduces governance attack surface.
Browser-only secret exposure	Zero private keys or deployer secrets	Required security boundary.
Automated test status	All unit, fuzz, invariant, parser and state-machine tests passing	Release gate.

Non-metrics for v1

- User count, volume, token price, TVL and speculative growth metrics are not product-completion criteria.
- Visual polish is not evaluated until the functional and security layers are stable and the frontend design gate is approved.
- Mainnet deployment is not a hackathon success requirement.

5. Scope and Non-Goals

V1 scope

- One registered active passkey per recipient wallet, with rotation and deactivation.
- Recipient-wallet EIP-712 / ERC-1271 signature fallback.
- Native MON payment requests only.
- Exact amount in wei.
- Expiry and maximum lifetime.
- Optional exact payer restriction.
- Short memo authenticated by hash.
- Random nonce/salt.
- Shareable URL-fragment payload and QR representation.
- Local verification before wallet connection.
- Onchain inspection before payment.
- Single-use settlement.
- Recipient revocation before payment.
- Passkey version invalidation after rotation/deactivation.
- Real inclusion and finality tracking.
- Permanent receipt from transaction/event/state.
- Local contact continuity and short request code.

Explicit non-goals

- Cross-chain payments or bridges.
- Token swaps or routing.
- Stablecoin or arbitrary ERC-20 support in the v1 contract.
- Escrow, refunds, disputes or chargebacks.
- Subscriptions or recurring mandates.
- Fiat on-ramp or off-ramp.
- Merchant SDK or payment processor integration.
- Backend signer, relayer or custodial wallet.
- Gas sponsorship as a core dependency.
- Social feed, chat, marketplace, DAO, rewards token or speculative economics.
- User identity verification, KYC or claims that a wallet equals a legal person.
- Upgradeable proxy, owner role, pause switch, admin sweep or protocol fee.

SCOPE DISCIPLINE Do not add features because they appear impressive. Any new feature must be justified by a user problem, a threat-model change, a real test and a three-minute demo contribution.

6. Actors, Trust Boundaries and Assumptions

Actor / Component	Trusted for	Not trusted for
Recipient wallet	Authorizing passkey registration, rotation, deactivation and revocation.	Legal identity, device security, honesty of memo.
Recipient passkey	Authenticating exact request terms for the registered wallet.	Spending funds, changing payout address, proving human identity.
Sender wallet	Signing the payment transaction.	Displaying accurate UI if the extension is compromised.
Vajra frontend	Constructing payloads and presenting data.	Creating success without chain evidence; storing secrets.
Vajra contract	Validating terms and enforcing one-time settlement.	Recovering mistakes, censoring scams or identifying people.
RPC provider	Returning chain state and receipts.	Perfect availability or freshness.
Browser / OS	Enforcing WebAuthn origin and user verification.	Freedom from malicious extensions or compromised devices.
Shared link / QR	Transporting the signed payload.	Confidentiality, authenticity or safe origin by itself.

Critical assumptions

- The recipient controls the wallet used to register the passkey.
- The browser and authenticator correctly implement WebAuthn.
- The production relying-party domain is securely controlled.
- The deployed contract address is configured from one canonical source.
- Current Monad network parameters and protocol support are re-verified before deployment.
- The sender independently confirms the Vajra origin and, for high-value payments, the Vajra Code through another channel.

7. User Journeys

Journey A - First-time recipient passkey setup

1. Connect recipient wallet on the configured Monad network.
2. Read a plain-language explanation: the passkey may authorize incoming payment requests but cannot spend wallet funds.
3. Create a WebAuthn credential with user verification required.
4. Locally verify the credential response and public key.
5. Submit a wallet-signed registration transaction containing the P-256 public key, credential identifier hash, relying-party identifier hash and initial key version.
6. Wait for successful transaction receipt and matching registration event.
7. Re-read the registry and show the passkey as active only after onchain confirmation.

Journey B - Recipient creates a request

1. Enter the exact MON amount, expiry, short memo and optional exact payer.
2. Read the latest chain timestamp and produce a bounded issuedAt / expiresAt window.
3. Generate a cryptographically random 32-byte nonce.
4. Construct the canonical request digest using the configured chain and contract domain.
5. Use WebAuthn to sign the digest as the challenge, or explicitly select wallet-signature fallback.
6. Verify the signature locally before generating the share artifact.
7. Create a versioned URL-fragment payload and QR code.
8. Display the full request terms, full recipient address, short Vajra Code and expiry.

Journey C - Sender verifies and pays

1. Open the request without connecting a wallet.
2. Strictly decode and validate the payload.
3. Verify chain ID, canonical contract, request digest and authentication mode.
4. Call the contract inspection function with a prospective payer address when known.
5. Display full recipient address, amount, expiry, memo, authentication method, passkey version and Vajra Code.
6. Connect a wallet only after the request is valid.
7. Switch to the configured Monad network if required.
8. Re-inspect and simulate immediately before payment.
9. Submit exact msg.value through fulfill.
10. Track transaction hash, receipt, event and finality.
11. Render a sealed receipt only after all success checks pass.

Journey D - Recipient revokes an unpaid request

1. Open a previously created request from local history or its share link.
2. Connect the recipient wallet.
3. Submit revoke with the exact request data.
4. Show that revocation remains pending until confirmed.
5. After confirmation, re-read request state and prevent payment.

Journey E - Lost device or passkey rotation

1. Connect the recipient wallet.
2. Register a replacement passkey and increment key version, or deactivate the active passkey.

3. Confirm the registry event and active version.
4. Treat every unpaid request signed by an older key version as invalid.
5. Keep wallet-signature fallback available for recovery.

8. Product Requirements - Functional

Request creation

- FR-001 [MUST]** The application MUST construct a canonical request containing recipient, optional payer, exact amount, issuedAt, expiresAt, random nonce, memoHash, authentication mode and authentication version.
- FR-002 [MUST]** The amount MUST be a positive integer number of wei and MUST NOT use JavaScript floating-point arithmetic.
- FR-003 [MUST]** The application MUST obtain issuedAt from recent chain state rather than relying only on the device clock.
- FR-004 [MUST]** The request lifetime MUST be at least 60 seconds and at most 30 days in v1.
- FR-005 [MUST]** The memo MUST be limited to 96 UTF-8 bytes after normalization.
- FR-006 [MUST]** The nonce MUST be 32 cryptographically random bytes and MUST NOT be derived from timestamp, amount or address.
- FR-007 [MUST]** The application MUST verify the generated request locally before it can be shared.
- FR-008 [MUST]** Request creation after passkey registration MUST require no transaction and no gas.

Authentication

- FR-009 [MUST]** Passkey mode MUST use a registered P-256 public key, WebAuthn user verification, the canonical request digest as challenge and the active key version.
- FR-010 [MUST]** Wallet fallback MUST use EIP-712 domain separation and MUST support ERC-1271 through SignatureChecker or an equivalent audited implementation.
- FR-011 [MUST]** The passkey MUST NOT authorize any function that transfers funds from the recipient wallet.
- FR-012 [MUST]** Rotating or deactivating a passkey MUST invalidate unpaid requests signed by earlier versions.
- FR-013 [MUST]** The frontend MUST label passkey authentication as registered-device authentication, not verified person or verified identity.

Sharing and verification

- FR-014 [MUST]** The request payload MUST be carried in the URL fragment rather than a query string.
- FR-015 [MUST]** The decoder MUST reject unknown versions, unknown keys, oversized payloads, malformed numbers, unsafe integers and invalid signatures.
- FR-016 [MUST]** The payment page MUST be readable and verifiable without wallet connection.
- FR-017 [MUST]** The payment page MUST display the full recipient address, exact amount, expiry, memo, payer restriction, chain, contract and request code.
- FR-018 [MUST]** The frontend MUST obtain chain name, explorer and contract metadata from trusted configuration, never from the shared payload.
- FR-019 [MUST]** Mutating any authenticated field MUST invalidate the request and remove the payment action.

Settlement

- FR-020 [MUST]** The sender MUST pay the exact signed amount using msg.value; underpayment and overpayment MUST revert.
- FR-021 [MUST]** A payer-restricted request MUST only be fulfillable by the exact signed payer address.
- FR-022 [MUST]** A request MUST settle at most once.
- FR-023 [MUST]** A revoked, expired, invalid, old-key-version or already-paid request MUST NOT display an enabled payment action.
- FR-024 [MUST]** The contract MUST update state before making the external native-MON transfer.
- FR-025 [MUST]** A failed recipient transfer MUST revert the entire transaction and leave the request unused.
- FR-026 [MUST]** The payment contract MUST NOT maintain user deposit balances.

Receipt and finality

- FR-027 [MUST]** The frontend MUST retain the transaction hash after broadcast even if the wallet disconnects or account changes.
- FR-028 [MUST]** Success MUST require receipt status success, a matching settlement event and a paid onchain state.
- FR-029 [MUST]** The finality seal MUST use observed chain state and MUST NOT display a hardcoded elapsed time.
- FR-030 [MUST]** When the finalized block tag or preferred finality mechanism is unavailable, the UI MUST clearly label the fallback confirmation method.

FR-031 [MUST] The permanent receipt MUST display sender, recipient, amount, request ID, memo hash, transaction hash, block number and settlement time.

Revocation and recovery

FR-032 [MUST] Only the recipient wallet MUST be able to revoke an unused request.

FR-033 [MUST] The UI MUST state that a submitted revocation is not effective until confirmed.

FR-034 [MUST] The wallet-signature fallback MUST remain available when the registered passkey is unavailable.

FR-035 [MUST] A lost-device recovery flow MUST require the recipient wallet, never only the old passkey.

9. State Machines

Request lifecycle

```
UNUSED
|-- fulfill(valid) --> PAID
|-- revoke(recipient wallet) --> REVOKED
```

EXPIRED is derived from block time.
 OLD_KEY_VERSION is derived from the active passkey registry.
 INVALID is derived from malformed data or failed authentication.

Frontend payment state

State	Primary action	Exit condition
Invalid payload	None	Open a valid request.
Invalid authentication	None	Obtain an untampered request.
Expired	None	Recipient creates a new request.
Revoked	None	Recipient creates a new request.
Already paid	View receipt	Inspect permanent settlement.
Wrong payer	None	Connect the permitted wallet.
Disconnected and valid	Connect wallet to pay	Wallet connected.
Wrong network	Switch to Monad	Network switch confirmed.
Ready	Pay exact amount	Simulation succeeds.
Awaiting wallet	Confirm in wallet	Wallet returns hash or rejection.
Broadcast	Payment submitted	Receipt available or transaction replaced.
Included	Waiting for finality	Finalized block covers transaction block.
Final	View sealed receipt	Terminal success.
Reverted / dropped / replaced	Retry or inspect replacement	State re-read and safe recovery.

Passkey lifecycle

```
NONE
|-- register(wallet) --> ACTIVE(version = 1)
ACTIVE(vN)
|-- rotate(wallet) --> ACTIVE(vN + 1)
|-- deactivate(wallet) --> INACTIVE(vN + 1)
INACTIVE
|-- register(wallet) --> ACTIVE(next version)
```

10. Frontend Design Decision Gate

THIS SECTION OVERRIDES CONVENIENCE The functional frontend may be built, but the coding agent **MUST** stop before any visual-design implementation and conduct a dedicated design conversation with the product owner. No Design.md is permitted before approval.

Allowed before the design conversation

- Routes, data loaders, hooks, contract interactions and transaction state machines.
- Semantic HTML structure and accessible labels.
- Unstyled or minimally browser-default forms and buttons.
- Simple responsive stacking required to test functionality.
- Error, loading, empty and success states in plain functional presentation.
- Automated tests, test IDs and story fixtures that do not establish a visual direction.
- A short document named docs/frontend-design-gate.md stating that design is pending.

Prohibited before approval

- Creating or modifying Design.md.
- Selecting a color palette, type system, logo, icon style, material language or visual metaphor.
- Choosing glassmorphism, brutalism, neo-brutalism, skeuomorphism, editorial minimalism or any other style category.
- Designing the landing page composition, hero treatment or background system.
- Creating production animations, scroll narratives or decorative microinteractions.
- Installing a design-system library because it is convenient.
- Using generated placeholder artwork as the permanent identity.
- Claiming visual approval based on agent preference or generic best practice.

Mandatory questions the coding agent must ask

1. What should Vajra feel like: institutional, futuristic, human, ritualistic, technical, calm, bold or another direction?
2. Which existing products, websites or physical objects should guide the aesthetic?
3. Which visual styles must be avoided?
4. Should the product be light-first, dark-first or offer both?
5. What role should the name Vajra play in the visual metaphor, if any?
6. How much motion is acceptable on landing, request creation, payment and receipt pages?
7. Should passkey authentication feel biometric, ceremonial, invisible, mechanical or something else?
8. What typography characteristics should the product use?
9. What mobile behavior is most important?
10. Are there existing brand assets, logos, icons or domain constraints?
11. Should the agent produce multiple visual directions, or one deeply researched direction?
12. Which screens must receive owner approval before design is propagated?

Required stop message

FUNCTIONAL FRONTEND GATE REACHED

The core Vajra flows are stable and tested. Visual design has intentionally not been selected. Before I create Design.md or implement production styling, I need to discuss the frontend direction with you.

Please answer the approved design questions in the project blueprint. I will not choose a visual system on your behalf.

Resume condition

Visual implementation may resume only after the product owner approves a written frontend direction. At that point, the coding agent may create Design.md and a separate visual implementation plan. This blueprint remains authoritative for product, security and technical behavior.

11. Smart Contract Architecture

Recommended contract

Vajra v1 should use one immutable contract named VajraNativeV1, with internal libraries for request hashing and WebAuthn verification. A single contract reduces deployment/configuration mistakes while preserving clear internal separation.

Contract properties

- Non-upgradeable.
- No owner or role-based administrator.
- No protocol fee.
- No pause switch.
- No arbitrary external calls.
- No delegatecall.
- No token approvals.
- No user-balance accounting.
- No admin sweep or withdrawal.
- Native MON only.

Core data structures

```
enum AuthMode { Wallet, Passkey }
enum RequestStatus { Unused, Paid, Revoked }

struct PaymentRequest {
    address recipient;
    address payer;           // zero address = any payer
    uint256 amount;          // wei
    uint64 issuedAt;
    uint64 expiresAt;
    bytes32 nonce;
    bytes32 memoHash;
    AuthMode authMode;
    uint32 authVersion;      // 0 for wallet mode; active key version for passkey mode
}

struct PasskeyCredential {
    bytes32 qx;
    bytes32 qy;
    bytes32 credentialIdHash;
    bytes32 rpIdHash;
    uint32 version;
    bool active;
}

struct Settlement {
    address payer;
    address recipient;
    uint256 amount;
    uint64 paidAt;
    bytes32 memoHash;
    AuthMode authMode;
    uint32 authVersion;
}
```

Storage

```
mapping(bytes32 requestId => RequestStatus) public statusOf;
mapping(bytes32 requestId => Settlement) public settlementOf;
mapping(address recipient => PasskeyCredential) public passkeyOf;
```

Canonical request digest

Wallet and passkey authentication must resolve to the same domain-separated request digest. The digest is the request identifier and replay-protection key.

```
structHash = keccak256(abi.encode(
    PAYMENT_REQUEST_TYPEHASH,
    recipient, payer, amount, issuedAt, expiresAt, nonce,
    memoHash, authMode, authVersion
));

requestId = keccak256(abi.encodePacked(
    "\x19\x01",
    domainSeparator(chainId, verifyingContract, name="Vajra", version="1"),
    structHash
));
```

External interface - draft

```
function requestId(PaymentRequest calldata request) external view returns (bytes32);

function inspect(
    PaymentRequest calldata request,
    AuthProof calldata proof,
    address prospectivePayer
) external view returns (bytes32 id, ValidationCode code);

function fulfill(
    PaymentRequest calldata request,
    AuthProof calldata proof
) external payable nonReentrant;

function revoke(PaymentRequest calldata request) external;

function registerPasskey(
    bytes32 qx, bytes32 qy, bytes32 credentialIdHash, bytes32 rpIdHash
) external;

function rotatePasskey(
    bytes32 qx, bytes32 qy, bytes32 credentialIdHash, bytes32 rpIdHash
) external;

function deactivatePasskey() external;
```


12. Passkey and Signature Architecture

Security boundary

PASSKEY AUTHORITY A passkey authorizes only an incoming payment request whose recipient is the wallet that registered the key. It cannot spend the recipient wallet balance, transfer tokens, change the payout address or manage wallet permissions.

WebAuthn proof requirements

- The authenticator data relying-party identifier hash must match the registered production RP ID hash.
- The user-presence bit must be set.
- The user-verification bit must be required for production requests.
- The client data type must be webauthn.get.
- The challenge must encode the canonical request digest exactly.
- The P-256 signature must verify against the active registered public key.
- The request authVersion must equal the active passkey version.
- The verifier must reject malformed offsets, malformed JSON, truncated authenticator data and oversized signatures.

Registration

Passkey registration is authorized by the recipient wallet. The contract may accept the selected public key without requiring an onchain attestation from the new key because the wallet transaction itself authorizes the binding. The frontend must still verify the WebAuthn registration response locally to prevent accidental registration of malformed data.

Rotation and deactivation

Every successful rotation or deactivation increments the credential version. Existing unpaid passkey requests carry the prior version and become invalid without requiring a per-request revocation transaction.

Wallet fallback

Wallet mode uses the same request digest and EIP-712 domain. SignatureChecker or an equivalent audited implementation should support EOAs and ERC-1271 smart wallets. Signature validity must be checked at fulfillment time and must never be cached as permanent truth.

Origin and RP-ID controls

- Production passkeys should use a stable application RP ID such as app.vajra.xyz or vajra.xyz after domain ownership is confirmed.
- Local development must use a separate environment and must never silently reuse production registration data.
- The contract registry should store rpIdHash per credential and compare it with authenticator data.
- The application must deploy HSTS, secure DNS and strict origin controls because passkey safety depends partly on domain control.

Signature modes

Mode	Strength	Use
Passkey	Human device authentication with onchain P-256 verification.	Flagship request-creation path.
Wallet EIP-712	Native wallet signature, including ERC-1271 where supported.	Fallback and recovery path.

13. Settlement Logic and Contract Safety

Fulfillment order

1. Validate static request fields and bounded time window.
2. Compute the canonical request ID.
3. Require request status Unused.
4. Require current block time not later than expiresAt.
5. Require authVersion and passkey state to be current when passkey mode is used.
6. Require msg.sender to equal the restricted payer when payer is non-zero.
7. Verify wallet or passkey authentication.
8. Require msg.value to equal amount exactly.
9. Set request status to Paid.
10. Write the settlement record.
11. Transfer native MON to the recipient with a checked low-level call.
12. Emit exactly one PaymentFulfilled event.

Checks-effects-interactions

```
statusOf[id] = RequestStatus.Paid;
settlementOf[id] = settlement;

(bool ok, ) = payable(request.recipient).call{value: request.amount}("");
if (!ok) revert NativeTransferFailed();

emit PaymentFulfilled(...);
```

Because a revert rolls back all state changes, a rejected native transfer leaves the request unused. fulfill must also be protected by nonReentrant.

Direct transfers

```
receive() external payable { revert DirectPaymentsDisabled(); }
fallback() external payable { revert DirectPaymentsDisabled(); }
```

Forced native transfers may still be possible at the EVM level. The contract must not expose a privileged sweep function merely to clean such funds. Documentation must state that Vajra does not intentionally hold user balances.

Revocation race

Payment and revocation may compete. Whichever transaction finalizes first determines the terminal state. The UI must not describe a pending revocation as effective.

Custom errors

```
error ZeroRecipient();
error ZeroAmount();
error InvalidNonce();
error MemoTooLong();
error InvalidTimeWindow();
error RequestExpired();
error RequestAlreadyPaid();
error RequestRevoked();
error WrongPayer(address expected, address actual);
error InvalidWalletSignature();
error InvalidPasskeyProof();
error InactivePasskey();
error WrongPasskeyVersion(uint32 expected, uint32 actual);
error IncorrectValue(uint256 expected, uint256 actual);
error NativeTransferFailed();
error DirectPaymentsDisabled();
```

14. Events, Receipts and Finality

Required events

```
event PasskeyRegistered(
    address indexed recipient, uint32 indexed version,
    bytes32 credentialIdHash, bytes32 rpIdHash
);

event PasskeyRotated(
    address indexed recipient, uint32 indexed version,
    bytes32 credentialIdHash, bytes32 rpIdHash
);

event PasskeyDeactivated(address indexed recipient, uint32 indexed version);

event PaymentFulfilled(
    bytes32 indexed requestId,
    address indexed payer,
    address indexed recipient,
    uint256 amount,
    uint64 paidAt,
    bytes32 memoHash,
    AuthMode authMode,
    uint32 authVersion
);

event PaymentRevoked(
    bytes32 indexed requestId,
    address indexed recipient,
    uint64 revokedAt
);
```

Receipt verification

The receipt layer must independently verify all available evidence before showing the final seal.

1. Transaction receipt status is success.
2. PaymentFulfilled event is emitted by the canonical Vajra contract.
3. Event requestId equals the locally computed request ID.
4. Event payer, recipient, amount, auth mode and version match the request and connected payer.
5. statusOf(requestId) returns Paid.
6. settlementOf(requestId) matches the event.
7. The transaction block is finalized according to the configured finality source.

Finality tracking

- Preferred path: query the finalized block tag and compare its height with the transaction block.
- Fallback path: use a documented confirmation strategy when the finalized tag is unavailable.
- Optional advanced demo path: use Monad Execution Events when dedicated node infrastructure is available.
- Never display a hardcoded 800 ms claim as the observed transaction time.
- If the RPC becomes unavailable after broadcast, retain the hash and present an honest verification-pending state.

Receipt states

State	Meaning
Broadcast	Wallet returned a transaction hash; no settlement claim yet.
Included	Successful receipt found in a block; finality not yet proven.
Final	Finalized chain state covers the settlement block.
Reverted	No successful payment event; funds did not settle through Vajra.
Replacement detected	Track the replacement transaction and verify it independently.
Verification delayed	Transaction may be successful, but one or more proof sources are temporarily unavailable.

15. Monad-Native Product Features

Vajra Touch

The flagship feature uses onchain P-256 verification for WebAuthn passkey assertions. Deployment must include a preflight test that confirms the expected P-256 verification path is active on the target Monad environment. The application must not assume protocol support merely because it existed in a prior environment.

Vajra Seal

The payment experience exposes real inclusion and finality as distinct states. This makes network performance visible as product feedback rather than a marketing claim.

Optional Execution Events mode

A richer live transaction heartbeat may be added when a Monad node and Execution Events sidecar are available. It is an enhancement, never a dependency. Standard RPC/WebSocket tracking must remain fully functional.

Deferred features

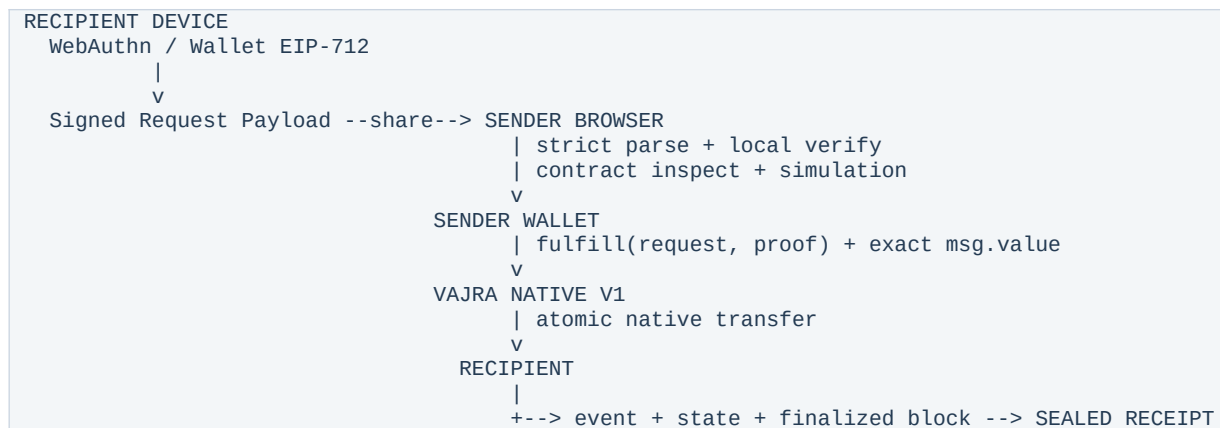
Feature	Decision	Reason
EIP-7702 one-action token payment	Deferred	Larger wallet delegation attack surface and not required for native MON.
Gas sponsorship	Deferred / opportunistic	Provider availability and eligibility are not protocol guarantees.
Reserve-aware maximum payment	Deferred	Requires current canonical interface validation and wallet-behavior research.
ERC-20 settlement	Separate later contract	Avoids non-standard token behavior in the v1 audit surface.
Multiple passkeys	Deferred	One active key plus rotation is easier to reason about for v1.

16. Technical Architecture

System components

Component	Responsibility
VajraNativeV1	Passkey registry, request inspection, single-use native settlement, revocation and receipts.
Next.js web application	Request creation, passkey ceremony, link/QR handling, wallet connection, payment and receipt rendering.
Typed-data library	Canonical request schema, domain, hashing and parity with Solidity.
WebAuthn library	Credential creation, assertion parsing, challenge generation and local verification.
Wallet integration	Monad network connection, wallet-signature fallback and payment transaction submission.
Public RPC layer	Reads, simulation, broadcast, receipt retrieval and finality tracking.
Optional indexer	Activity history and discovery; never authoritative for payment validity.
Optional Execution Events sidecar	Low-latency enriched transaction lifecycle for advanced demo/operations.

End-to-end architecture



Technology choices

Layer	Recommended choice	Constraint
Contracts	Solidity with Foundry	Pin compiler and dependency versions.
Libraries	Current stable OpenZeppelin releases appropriate to compiler	Do not copy unaudited cryptographic code.
Web	Next.js App Router + TypeScript	No visual system selected yet.
Wallet	wagmi/viem or equivalent mature stack	One canonical chain and contract configuration.
Passkeys	Browser WebAuthn APIs plus audited verification utilities	No custom cryptography.
Schema validation	Strict runtime schema library	Reject unknown keys and malformed integers.
Testing	Foundry + unit/integration/E2E browser tests	Fuzz and invariant testing are mandatory.

17. Canonical Data Model and Share Payload

Payment request

Field	Type	Rule
recipient	address	Non-zero; receives MON.
payer	address	Zero means open request; otherwise exact allowed payer.
amount	uint256	Positive exact wei.
issuedAt	uint64	Derived from recent chain timestamp.
expiresAt	uint64	Strictly after issuedAt and within max lifetime.
nonce	bytes32	Cryptographically random and non-zero.
memoHash	bytes32	keccak256 of normalized UTF-8 memo.
authMode	enum	Wallet or Passkey.
authVersion	uint32	0 for wallet; active passkey version for passkey.

Versioned envelope

```
{
  "v": 1,
  "chainId": "<decimal string>",
  "verifyingContract": "0x...",
  "request": {
    "recipient": "0x...",
    "payer": "0x0000000000000000000000000000000000000000000000000000000000000000",
    "amount": "1000000000000000000",
    "issuedAt": "<unix seconds>",
    "expiresAt": "<unix seconds>",
    "nonce": "0x...",
    "memo": "Dinner",
    "authMode": "passkey",
    "authVersion": "1"
  },
  "proof": {
    "kind": "webauthn",
    "authenticatorData": "<base64url>",
    "clientDataJSON": "<base64url>",
    "challengeIndex": 0,
    "typeIndex": 0,
    "r": "0x...",
    "s": "0x..."
  }
}
```

Transport rules

- Use `https://vajra.xyz/pay#<base64url-payload>` after domain control is confirmed.
- Maximum encoded payload: 8 KB unless tests establish a lower safe limit for QR reliability.
- Maximum decoded WebAuthn proof size: explicit bounded limit.
- No unknown fields, nested arbitrary objects or executable data.
- Numeric values remain decimal strings until converted with checked bigint parsing.
- Memo is displayed as plain text only.
- No analytics, session replay or third-party widgets on payment routes.

Vajra Code

Derive a short human-comparison code from the request ID. The code is a usability aid, not a cryptographic replacement. It should be shown identically to recipient and sender and may be confirmed through a separate channel for high-value payments.

18. Frontend Functional Architecture

Routes

Route	Purpose
/	Product explanation and entry points; visual design pending.
/request	Create, authenticate and share a payment request.
/pay#payload	Verify and fulfill a shared request.
/receipt/[requestId]	Render a permanent public settlement receipt.
/activity	Show real locally known or indexed requests and payments.
/settings/passkey	Register, rotate and deactivate the device passkey.
/api/health	Deployment and RPC health signal without secrets.

Recommended module layout

```

apps/web/
  app/
    page.tsx
    request/page.tsx
    pay/page.tsx
    receipt/[requestId]/page.tsx
    activity/page.tsx
    settings/passkey/page.tsx
    api/health/route.ts
  features/
    passkey-registration/
    create-request/
    verify-request/
    fulfill-request/
    revoke-request/
    receipt/
    activity/
  lib/
    vajra/domain.ts
    vajra/types.ts
    vajra/hash.ts
    vajra/encode.ts
    vajra/decode.ts
    vajra/fingerprint.ts
    webauthn/
    contracts/
    chain/
    wallet/
    finality/
    errors/
    security/
  components/
    functional/    # semantic, unstyled before design gate
  docs/
    frontend-design-gate.md

```

Architecture rules

- Business logic must not live inside presentation components.
- TypeScript and Solidity must share one canonical request-field order and type definition.
- Contract addresses, chain ID, explorer URLs and RPC endpoints must come from one configuration module.
- Production configuration must fail closed when the contract address is missing or invalid.
- The request parser must be pure, deterministic and independently tested.
- Wallet/account/network changes must invalidate simulations and preflight results.
- Transaction hash recovery must survive refresh where feasible without inventing success.

19. Security Threat Model

Threat	Impact	Required mitigation
Amount or recipient mutation	Sender pays unintended terms.	All material fields included in the request digest; local and onchain verification.
Cross-chain replay	Signature reused on another chain.	chainId in EIP-712 domain and canonical chain allowlist.
Cross-contract replay	Signature reused in another deployment.	verifyingContract in domain.
Duplicate fulfillment	Sender pays twice.	Persistent status keyed by request ID; state check before transfer.
Wrong payer / front-run	Unauthorized wallet fulfills a restricted request.	payer field signed and msg.sender enforced.
Complete valid-request substitution	Scammer supplies their own correctly signed request.	Full address, local continuity warning and optional out-of-band Vajra Code confirmation.
Address poisoning	User copies a look-alike history address.	No manual destination entry in the pay flow.
Passkey compromise	Attacker creates incoming requests for registered recipient.	Passkey cannot spend recipient funds or change payout address; wallet rotation/deactivation.
Lost passkey	Recipient cannot create passkey requests.	Wallet fallback and wallet-authorized rotation.
Old key replay	Request created before rotation remains payable.	authVersion must equal current active version.
WebAuthn origin abuse	Credential used outside intended app context.	RP-ID hash verification, secure domain control, HSTS and browser-origin enforcement.
Reentrancy	Recipient callback attempts duplicate action.	Checks-effects-interactions and nonReentrant.
Recipient rejects MON	Payment status becomes inconsistent.	Revert full transaction; no state remains paid.
Malformed payload / DOM XSS	Browser compromise or misleading UI.	Strict schema, size bounds, text-only rendering, CSP.
RPC stale or unavailable	False state or false success.	Multiple reads, simulation, event/state verification, honest unavailable states.
Wallet account/chain switch	Wrong payer or stale simulation.	Invalidate and re-run inspection/simulation.
Compromised frontend origin	User sees malicious transaction.	Canonical domain verification, CSP, verified contract, wallet confirmation, open-source reproducible deployment.
Admin compromise	Funds redirected or contract altered.	No admin, no upgradeability, no fee receiver.

20. Frontend and Browser Security

Untrusted input handling

- Treat every fragment byte as hostile until strict parsing succeeds.
- Never use dangerouslySetInnerHTML, innerHTML, eval, Function or document.write.
- Render memo and labels as plain text.
- Never construct explorer, RPC or redirect URLs from payload fields.
- Reject control characters and confusing Unicode where they reduce safe comparison.
- Show full addresses in expandable or copy-safe form; truncation alone must not be the only representation.

Security headers

```
Content-Security-Policy:
  default-src 'self';
  script-src 'self' 'nonce-<per-response>';
  style-src 'self';
  img-src 'self' data;
  font-src 'self';
  connect-src 'self' <approved RPC/WSS endpoints>;
  object-src 'none';
  base-uri 'none';
  frame-ancestors 'none';
  form-action 'self';

Referrer-Policy: no-referrer
X-Content-Type-Options: nosniff
Permissions-Policy: camera=(self), microphone=(), geolocation=()
Cross-Origin-Opener-Policy: same-origin
```

Payment transaction pipeline

1. Verify the payload locally.
2. Read contract state.
3. Connect the sender wallet.
4. Validate account and chain.
5. Inspect the request again with the actual payer.
6. Simulate fulfill using exact msg.value.
7. Display exact recipient and amount immediately before wallet confirmation.
8. Request transaction signature.
9. Persist transaction hash after broadcast.
10. Wait for receipt and decode event.
11. Re-read request status and settlement.
12. Track finality and render the final seal.

21. Security Invariants

The following properties are release blockers. Invariant tests should exercise randomized action sequences and malicious recipients.

1. A request can settle at most once.
2. A paid request can never become revoked or unused.
3. A revoked request can never become paid.
4. Successful settlement transfers exactly the signed amount.
5. Successful settlement transfers only to the signed recipient.
6. A restricted request can only be settled by the signed payer.
7. An expired request never transfers MON.
8. An invalid signature or WebAuthn proof never transfers MON.
9. A passkey request with an inactive or stale key version never transfers MON.
10. A failed recipient transfer leaves the request unused.
11. Reentrant execution cannot create a second settlement or corrupt state.
12. No privileged account can change a request, redirect a payment or recover funds.
13. No function can execute arbitrary user-supplied calldata.
14. The TypeScript request ID always equals the Solidity request ID for the same canonical fields.
15. Every successful fulfillment emits exactly one matching PaymentFulfilled event.
16. The frontend never renders Paid before independent receipt, event and state checks succeed.

22. Testing Strategy

Contract unit tests

- Valid wallet-authenticated open request.
- Valid wallet-authenticated payer-restricted request.
- Valid passkey-authenticated request.
- ERC-1271 wallet signature.
- Correct settlement record and event.
- Registration, rotation and deactivation.
- Revocation by recipient.
- Inspection result for every validation code.

Contract revert tests

- Zero recipient, zero amount, zero nonce and invalid time window.
- Memo hash mismatch where plaintext memo is supplied for verification.
- Expired request and future issuedAt beyond tolerance.
- Invalid wallet signature, wrong chain domain and wrong contract domain.
- Malformed WebAuthn proof, wrong challenge, wrong RP-ID hash, absent user verification and invalid P-256 signature.
- Inactive passkey and stale key version.
- Wrong payer, underpayment and overpayment.
- Duplicate payment, revoked request and duplicate revocation.
- Revocation by non-recipient.
- Recipient that rejects MON.
- Reentrant recipient.
- Direct receive and fallback transfer.

Fuzz tests

- Amounts, timestamps, lifetime boundaries and msg.value.
- Recipients, payers and nonces.
- Memo bytes and normalization boundaries.
- Signature and WebAuthn proof byte lengths.
- Random action sequences across register, rotate, deactivate, revoke and fulfill.

Differential parity tests

RELEASE BLOCKER For thousands of randomized requests, the TypeScript struct hash, EIP-712 digest and request ID must exactly equal Solidity results. Any mismatch blocks deployment.

Malicious test contracts

```
ReentrantRecipient
RejectingRecipient
GasHeavyRecipient
ValidERC1271Wallet
RevocableERC1271Wallet
AlwaysInvalidERC1271Wallet
RevertingERC1271Wallet
```

Frontend tests

- Strict payload parsing and size limits.
- Malformed base64url, unknown versions, missing and extra keys.
- Decimal-string bigint parsing and rejection of scientific notation.
- Tampered amount, recipient, payer, expiry, memo, mode and key version.
- Wallet rejection, account switch, chain switch, disconnect and replacement transaction.
- Duplicate click prevention and contract-level duplicate failure.
- RPC unavailable, stale read and delayed receipt refresh.
- Refresh after broadcast and transaction-hash recovery.
- Reduced motion, keyboard operation and screen-reader status updates after design implementation.

End-to-end testnet tests

1. Register a real test passkey.
2. Create and verify a passkey request.
3. Pay real native MON.
4. Verify recipient balance and sealed receipt.
5. Attempt duplicate fulfillment.
6. Modify one field and confirm invalidation.
7. Create and revoke another request.
8. Create a payer-restricted request and test wrong and correct payers.
9. Rotate the passkey and confirm the old unpaid request is invalid.
10. Exercise wallet-signature fallback.

23. Error Handling and Recovery

User-facing error standard

Every transaction error must answer four questions: what happened, whether funds moved, whether retry is safe, and how the user can independently verify the state.

Condition	Primary copy requirement
Wallet rejected	No transaction was sent. You can retry safely.
Simulation failed	Payment was not sent. Show the specific actionable reason without raw stack output.
Broadcast hash obtained	Payment is submitted; keep the hash even if the wallet disconnects.
Transaction reverted	The Vajra payment did not settle. Provide explorer link and safe retry guidance after re-inspection.
RPC unavailable	Do not infer failure or success. Preserve known transaction data and offer verification retry.
Event decode failure with successful receipt	State that the transaction succeeded but receipt verification is incomplete; re-read contract state.
Already paid	Do not offer another payment. Link to the existing settlement.
Stale passkey version	Recipient must create a new request using the active credential.
Wrong payer	This request is addressed to another wallet; no payment action.

Raw errors

Raw RPC and Solidity errors may be logged locally in development but must not be the primary production copy. Error classification should map known wallet, transport, revert and replacement conditions to stable application error codes.

24. Observability, Privacy and Data Integrity

Observability

- Health endpoint reports build version, configured chain, contract-address presence and RPC reachability without secrets.
- Client logs must never contain private keys, seed phrases, full WebAuthn credential material beyond necessary public proof, or unredacted sensitive payloads.
- Production error reporting must redact URL fragments and payment details by default.
- Every final receipt should be reproducible from public chain data and the shared request payload.

Privacy

- Payment sender, recipient and amount are public onchain.
- Shared links reveal request data to anyone who receives them.
- The plaintext memo is transported in the fragment but only its hash is stored/emitted onchain.
- No analytics, advertisements or session replay tools are allowed on request and payment routes.
- Contact continuity is local convenience data and should not be uploaded by default.

Activity data

Activity may be derived from local request history, wallet event queries or an indexer. The indexer is never authoritative for validity. The UI must not invent events, analytics, volume, users or success metrics.

25. Deployment and Environment Requirements

Environments

Environment	Purpose	Rules
Local	Unit, parser, contract and browser development.	Separate WebAuthn RP configuration and local test keys.
Monad test environment	Real contract, passkey and wallet QA.	Canonical addresses and explorer links; no mainnet claims.
Production testnet demo	Hackathon/public demo deployment.	Verified source, reproducible build, secure headers and clean repository.
Mainnet	Out of scope.	Blocked until independent audit and explicit release decision.

Environment variables

```

NEXT_PUBLIC_VAJRA_CHAIN_ID
NEXT_PUBLIC_VAJRA_CONTRACT_ADDRESS
NEXT_PUBLIC_VAJRA_EXPLORER_URL
NEXT_PUBLIC_VAJRA_RP_ID
NEXT_PUBLIC_VAJRA_ORIGIN
VAJRA_RPC_URL
DEPLOYER_PRIVATE_KEY    # deployment only; never exposed to browser

```

Deployment gates

- Exact Solidity compiler version pinned.
- OpenZeppelin and all dependencies pinned in lockfile.
- Optimizer settings recorded.
- Deployment script dry-run completed.
- P-256/WebAuthn verification preflight passes on target Monad environment.
- Contract source and bytecode verified on the explorer.
- Canonical contract address written once and consumed everywhere.
- Production build fails closed without valid configuration.
- No localhost, placeholder contract, demo fallback or private key in the build.
- All tests pass from a clean clone.

26. Delivery Phases

Phase	Goal	Exit criteria
0 - Specification freeze	Approve PRD, TRD, threat model and design gate.	No unresolved contract-schema or scope questions.
1 - Contract core	Implement hashing, passkey registry, wallet/passkey verification, fulfill, revoke and events.	Unit, fuzz and invariant tests pass.
2 - Functional web	Build unstyled semantic request, pay, receipt and passkey flows.	Real testnet request and payment work end to end.
3 - Security hardening	Parser hardening, CSP, race handling, RPC/finality resilience and E2E adversarial tests.	Security checklist and differential parity complete.
4 - Frontend design gate	Stop and conduct owner design discussion.	Written design direction approved; Design.md may then be created.
5 - Visual implementation	Apply approved design system without changing business logic.	Owner approval, accessibility and performance checks.
6 - Submission readiness	README, verified deployment, demo fixture, demo script and clean clone.	Three-minute demo succeeds without fake data.

No phase skipping

The coding agent must not move visual design earlier because a functional page looks plain. Plain is acceptable before Phase 4; premature design is not.

27. Coding Agent Operating Instructions

AUTHORITATIVE INSTRUCTION The coding agent must follow this section even when convenience, generated templates or previous project habits suggest otherwise.

Before coding

- Read the entire blueprint.
- Create an implementation plan that maps every MUST requirement to code and tests.
- List every unresolved assumption instead of inventing a value.
- Confirm current Monad network configuration and P-256 support before deployment.
- Do not reuse code, design or product claims from unrelated earlier projects.

During coding

- Keep contracts minimal and immutable.
- Use audited libraries for EIP-712, ERC-1271, WebAuthn and P-256 verification.
- Do not duplicate request hashing logic.
- Write tests with each functional unit, not after the application is complete.
- Do not create fake success, demo-only fallback or hardcoded transaction state.
- Do not store or expose private keys.
- Do not install large visual libraries before the design gate.

Required frontend stop

After functional flows and security tests are stable, the coding agent must stop. It must not create Design.md or production styling. It must ask the mandatory design questions in Section 8 and wait for the owner.

Completion report format

- Commit hashes.
- Files changed.
- Requirements implemented.
- Tests added and exact pass counts.
- Real testnet transaction hashes where applicable.
- Security findings and unresolved risks.
- Git status.
- Design-gate status.
- Remaining blockers.

28. Definition of Done

Vajra v1 is functionally complete only when every item below is true.

- ☐ Recipient wallet can register, rotate and deactivate one active passkey.
- ☐ Passkey request creation works without a transaction after registration.
- ☐ Wallet EIP-712 / ERC-1271 fallback works.
- ☐ Sender can verify a request without connecting a wallet.
- ☐ Every authenticated field is displayed before payment.
- ☐ Modifying any authenticated field invalidates the request.
- ☐ Wrong chain and wrong contract fail.
- ☐ Wrong payer fails.
- ☐ Expired request fails.
- ☐ Revoked request fails.
- ☐ Stale passkey-version request fails.
- ☐ First valid payment transfers exact MON to the recipient.
- ☐ Second payment attempt fails at contract level.
- ☐ Recipient balance changes from a real testnet transaction.
- ☐ Receipt is derived from real receipt, event and contract state.
- ☐ Finality seal is based on observed chain state.
- ☐ Contract has no owner, upgrade, fee, admin withdrawal or user balance accounting.
- ☐ All unit, fuzz, invariant, parser, state-machine and E2E tests pass.
- ☐ Contract source is verified.
- ☐ Production build passes from a clean clone.
- ☐ No secrets or generated build artifacts are committed.
- ☐ Design.md does not exist until the owner approves the design direction.

29. Residual Risks and Honest Limitations

1. A scammer can create a valid request for their own wallet. Vajra verifies control and integrity, not morality or legal identity.
2. A compromised recipient wallet can register an attacker-controlled passkey.
3. A compromised sender wallet or malicious browser extension can approve an unwanted transaction.
4. A phishing clone can imitate the frontend; users must verify the canonical domain and contract.
5. The public blockchain exposes sender, recipient, amount and timing.
6. Anyone with the share link can read its contents.
7. RPC providers may fail, lag or disagree temporarily.
8. Smart-wallet signature validity may change according to that wallet contract.
9. Recipient contracts may reject native MON, causing fulfillment to revert.
10. No test suite or audit can prove the complete absence of unknown vulnerabilities.

SECURITY LANGUAGE Never market Vajra as loophole-free, impossible to hack or guaranteed safe. Correct language is: minimized attack surface, explicit security boundaries, tested invariants, immutable settlement logic and independently verifiable results.

30. Open Decisions Before Implementation

Decision	Owner action	Default if unresolved
Confirmed control of vajra.xyz	Verify registrar and DNS access.	Use a local/test domain and do not publish the domain claim.
Production RP ID	Select vajra.xyz or app.vajra.xyz.	Do not register production passkeys.
Target Monad environment	Confirm current official chain ID, RPC and explorer.	Block deployment.
Open-request front-running policy	Approve zero-address payer behavior.	Allow because attacker can only pay the signed recipient; document it.
Memo normalization	Approve NFC normalization and control-character policy.	NFC plus strict control-character rejection.
Maximum QR payload	Measure across target devices.	Use share link when QR exceeds tested bound.
Activity indexing	Local history only or optional indexer.	Local history plus direct event reads.
Frontend design direction	Complete dedicated design discussion.	No production styling and no Design.md.

Appendix A - EIP-712 Schema and Digest Parity

```
EIP712Domain(  
  string name,  
  string version,  
  uint256 chainId,  
  address verifyingContract  
)  
  
PaymentRequest(  
  address recipient,  
  address payer,  
  uint256 amount,  
  uint64 issuedAt,  
  uint64 expiresAt,  
  bytes32 nonce,  
  bytes32 memoHash,  
  uint8 authMode,  
  uint32 authVersion  
)
```

The field order, integer widths and enum encodings must be identical in Solidity, TypeScript, tests and documentation. A single canonical module should generate typed data for wallet mode and the 32-byte challenge digest for passkey mode.

Recommended parity vectors

Vector	Purpose
All-zero optional payer	Open request baseline.
Exact payer	Payer restriction.
Minimum amount	Boundary handling.
Maximum uint256 amount in unit test	Integer serialization safety.
Minimum and maximum lifetime	Timestamp boundaries.
Unicode memo after normalization	Cross-language memoHash parity.
Wallet and passkey auth modes	Enum/version parity.

Appendix B - Validation Codes

Code	Meaning
Valid	Request is currently payable by the prospective payer.
ZeroRecipient	Recipient is the zero address.
ZeroAmount	Amount is zero.
InvalidNonce	Nonce is zero or malformed.
InvalidTimeWindow	issuedAt/expiresAt violate bounds.
Expired	Current block time is after expiresAt.
AlreadyPaid	Request status is Paid.
Revoked	Request status is Revoked.
WrongPayer	Prospective payer does not match the restriction.
InactivePasskey	Recipient has no active passkey for passkey mode.
WrongPasskeyVersion	Request version does not equal active version.
InvalidWalletSignature	EIP-712 / ERC-1271 signature failed.
InvalidPasskeyProof	WebAuthn/P-256 proof failed.
IncorrectValue	msg.value does not equal amount.

Appendix C - Security Review Checklist

- ☐ No owner, roles, proxy, delegatecall or admin withdrawal.
- ☐ No tx.origin authorization.
- ☐ State is updated before native transfer.
- ☐ nonReentrant protects fulfill.
- ☐ Exact msg.value is enforced.
- ☐ Cross-chain and cross-contract domain separation verified.
- ☐ Wallet and passkey proofs share the same request digest.
- ☐ RP-ID hash, user presence and user verification checked.
- ☐ Passkey version invalidates old unpaid requests.
- ☐ No custom cryptography where audited libraries exist.
- ☐ Strict parser rejects unknown and oversized data.
- ☐ No DOM-XSS sinks.
- ☐ CSP and no-referrer deployed.
- ☐ Account/chain changes invalidate simulation.
- ☐ Success requires receipt + event + state.
- ☐ TypeScript/Solidity parity corpus passes.
- ☐ Fuzz and invariant campaigns complete.
- ☐ Testnet extension-wallet QA complete.
- ☐ Contract source and bytecode verified.
- ☐ Independent review completed before any mainnet discussion.

Appendix D - Reference Set

The following standards and security references informed this blueprint. Network-specific values and protocol activation must be re-verified against current official Monad documentation before deployment.

- R1. EIP-712: Ethereum typed structured data hashing and signing: <https://eips.ethereum.org/EIPS/eip-712>
- R2. EIP-1271: Standard signature validation method for contracts: <https://eips.ethereum.org/EIPS/eip-1271>
- R3. EIP-7951: P-256 verification precompile: <https://eips.ethereum.org/EIPS/eip-7951>
- R4. EIP-7702: Set EOA account code for one transaction / delegation model: <https://eips.ethereum.org/EIPS/eip-7702>
- R5. OpenZeppelin cryptography and WebAuthn documentation: <https://docs.openzeppelin.com/contracts/5.x/api/utils/cryptography>
- R6. Solidity security considerations: <https://docs.soliditylang.org/en/latest/security-considerations.html>
- R7. Foundry invariant testing: <https://getfoundry.sh/forge/invariant-testing>
- R8. OWASP DOM-based XSS prevention cheat sheet: https://cheatsheetseries.owasp.org/cheatsheets/DOM_based_XSS_Prevention_Cheat_Sheet.html
- R9. MDN Content Security Policy implementation guidance: https://developer.mozilla.org/en-US/docs/Web/Security/Practical_implementation_guides/CSP
- R10. Monad developer documentation and current network information: <https://docs.monad.xyz/>

Verification note

This PDF is a build specification, not an audit report. Before deployment, verify current compiler support, OpenZeppelin APIs, Monad chain configuration, P-256 verification behavior, finalized-block semantics and explorer endpoints in the exact target environment.

END OF AUTHORITATIVE VAJRA V1 BLUEPRINT